

UIDE COMPLET HADOOP 3.3.6

PARTIE 1 — Big Data & Pourquoi Hadoop

Diapo 3 — Pourquoi parle-t-on de Big Data ?

Aujourd'hui, chaque action numérique génère des données : un like sur Facebook, une transaction bancaire, un capteur IoT qui envoie sa température toutes les secondes. La quantité produite dépasse ce que les bases classiques (MySQL, Oracle) peuvent gérer. C'est là qu'on parle de **Big Data**.

V	Signification	Exemple
Volume	Quantité massive	Des pétaoctets de logs serveur
Vélocité	Vitesse de génération	1 million de tweets par minute
Variété	Formats hétérogènes	JSON, vidéos, textes, capteurs
Véracité	Fiabilité des données	Données bruitées ou corrompues
Valeur	Utilité pour la décision	Les recommandations Netflix

Comment présenter cette diapo : Commence par une question : *"Combien d'emails sont envoyés par minute dans le monde ?"* (réponse : ~300 millions). Puis dis que les systèmes classiques ne peuvent pas stocker et traiter autant simultanément.

Diapo 4 — Limites des systèmes traditionnels

1. Stockage centralisé — Toutes les données sont sur UN seul serveur. Si ce serveur tombe, tout s'arrête. C'est le fameux **Single Point of Failure (SPOF)**.

2. Scalabilité verticale — Pour plus de capacité, on ajoute du CPU/RAM au même serveur. Mais un serveur a des limites physiques et un prix exorbitant.

3. Coût élevé — Les serveurs haute performance coûtent des centaines de milliers d'euros.

4. Single Point of Failure — Dans le Big Data, une seule panne = système complètement indisponible.

Comment présenter : Utilise cette analogie : *"Imagine une bibliothèque avec un seul bibliothécaire. Plus il y a de lecteurs, plus c'est lent. Si le bibliothécaire est malade, tout s'arrête. Hadoop c'est une bibliothèque avec des dizaines de bibliothécaires qui travaillent en parallèle."*

PARTIE 2 — Architecture Hadoop

Diapo 6 — Qu'est-ce qu'Hadoop ?

Hadoop est un framework open-source Apache conçu pour stocker et traiter de très grandes quantités de données sur un **cluster de machines ordinaires** (pas des super-serveurs). L'idée fondatrice : **diviser pour régner** — on découpe les données en blocs, on les distribue sur plusieurs machines, et chaque machine traite sa part en parallèle.

Composant	Rôle	Analogie
HDFS	Système de fichiers distribué — stockage	Disque dur partagé entre toutes les machines
YARN	Gestionnaire de ressources	Chef de chantier qui distribue le travail
MapReduce	Modèle de calcul distribué	Méthode de travail en 3 étapes

Comment présenter : *"Hadoop c'est comme Google Drive mais pour des entreprises qui traitent des milliards de fichiers, avec des centaines de disques durs qui travaillent tous en même temps."*

Diapo 7 — Pourquoi choisir Hadoop ?

01 — Gestion des données massives : Hadoop gère ce que les systèmes classiques ne peuvent pas — des téraoctets, voire des pétaoctets. Netflix et Facebook en sont les exemples parfaits.

02 — Scalabilité horizontale : Au lieu d'acheter un serveur plus puissant (cher et limité), on ajoute simplement de nouvelles machines ordinaires au cluster. Hadoop s'adapte automatiquement.

03 — Coût-efficacité : Hadoop utilise du **commodity hardware** (matériel standard). On peut construire un cluster avec des serveurs à 1 000€ au lieu de machines spécialisées à 100 000€.

Diapo 8 — Cas d'utilisation : Netflix & Facebook

Netflix	Facebook
Analyse des comportements de visionnage	Analyse de milliards d'interactions/jour
Recommandation de films (ML sur HDFS)	Détection de contenus populaires
Détection des tendances	Publicité ciblée
Optimisation qualité streaming	Détection de comportements suspects
Chaque clic → données HDFS → traitement distribué → recommandation personnalisée	Likes, partages, commentaires → Hadoop → compréhension des intérêts utilisateurs

PARTIE 3 — Composants actifs (commande `jps`)

Quand on lance Hadoop avec `start-dfs.sh` et `start-yarn.sh`, plusieurs processus Java démarrent. La commande `jps` (Java Process Status) les liste tous. La diapo 30 montre exactement :

```
$ jps
8992 SecondaryNameNode
11217 ResourceManager
10625 FsShell
8779 DataNode
8651 NameNode
11339 NodeManager
11676 Jps
```

Voici le rôle précis de chacun :

NameNode — Le cerveau de HDFS

Le NameNode **ne stocke PAS les données**. Il stocke uniquement les **métadonnées** : où est quel fichier, sur quel DataNode, découpé en combien de blocs. C'est l'annuaire du système de fichiers distribué.

- Il tient à jour deux fichiers : `fsimage` (snapshot du système) et `edits` (journal des modifications)
- Si le NameNode tombe → plus personne ne sait où sont les données → cluster inutilisable
- C'est pourquoi on le sauvegarde régulièrement et en production on utilise le mode **High Availability**

Analogie : Le NameNode est comme le **sommaire d'un livre**. Il ne contient pas le texte, mais il te dit "le chapitre 3 est à la page 47". Les pages (les données), c'est le DataNode qui les a.

DataNode — Les muscles de HDFS

Les DataNodes **stockent physiquement les blocs de données**. Dans un vrai cluster, il y en a des dizaines. Ici, on en a 1 car on est en mode pseudo-distribué (une seule machine).

- Chaque DataNode envoie un **heartbeat** (battement de cœur) au NameNode toutes les 3 secondes pour dire "je suis vivant"
 - Si le NameNode ne reçoit plus de heartbeat → il marque le DataNode comme mort → il réplique ses blocs sur d'autres nœuds
 - La diapo 33 montre le heartbeat : "Last Heartbeat : 1s" (le nœud est en bonne santé)
-

SecondaryNameNode — L'assistant du NameNode

Attention : ce n'est PAS un backup du NameNode ! C'est une erreur de naming classique. Son rôle réel :

- Il fusionne périodiquement `fsimage` + `edits` → produit un nouveau `fsimage` à jour
 - Ça évite que le fichier `edits` grossisse indéfiniment et ralentisse le NameNode
 - En production moderne, on utilise plutôt le **JournalNode** avec la High Availability
-

ResourceManager — Le patron de YARN

Le ResourceManager est **le chef d'orchestre de tout le cluster de calcul**. Il :

- Reçoit les demandes d'exécution de jobs (ex: lancer notre JAR MapReduce)
 - Alloue les ressources (CPU, mémoire) aux applications
 - Surveille l'avancement global
 - Est accessible via l'interface web `localhost:8088`
-

NodeManager — L'agent local de YARN

Le NodeManager est **l'agent de terrain** sur chaque machine du cluster. Il :

- Signale au ResourceManager les ressources disponibles sur sa machine
 - Exécute les **conteneurs** (les tâches Map et Reduce)
 - Surveille l'utilisation mémoire/CPU des conteneurs et les tue si nécessaire
-

ApplicationMaster — Le chef de projet d'un job

Pour chaque job soumis, YARN lance un **ApplicationMaster** dédié. Il :

- Négocie les ressources nécessaires avec le ResourceManager
 - Surveille l'avancement de ses tâches Map et Reduce
 - Relance les tâches échouées
 - Disparaît quand le job est terminé
-

Flux YARN complet quand on lance un job :

Client → ResourceManager (soumet le job)

↓

ResourceManager → choisit un NodeManager → lance ApplicationMaster

↓

ApplicationMaster → négocie des conteneurs avec ResourceManager

↓
NodeManager × N → exécute les conteneurs (tâches Map/Reduce)
↓
ApplicationMaster → surveille → signale SUCCEEDED au ResourceManager

HDFS en détail — La réplication

Quand tu uploades un fichier dans HDFS, il est découpé en **blocs de 128 Mo** (par défaut). Chaque bloc est **répliqué sur plusieurs DataNodes**.

Pourquoi 3 réplications (diapo 22) ?

- 1 copie = risque total si le DataNode tombe
- 2 copies = encore vulnérable si 2 pannes simultanées
- 3 copies = équilibre optimal sécurité / coût de stockage
- Au-delà de 3 = trop coûteux pour un gain de fiabilité faible

En mode pseudo-distribué (1 seule machine), on met `dfs.replication = 1` car on n'a qu'un seul DataNode.

PARTIE 4 — Configuration pas à pas

Les 4 fichiers de configuration (diapos 18-25)

Tous dans `/usr/local/hadoop-3.3.6/etc/hadoop/`

Fichier	Rôle
<code>hadoop-env.sh</code>	Variables d'environnement shell
<code>core-site.xml</code>	Configuration globale HDFS
<code>hdfs-site.xml</code>	Configuration du système de fichiers
<code>mapred-site.xml</code>	Configuration MapReduce
<code>yarn-site.xml</code>	Configuration YARN

hadoop-env.sh (diapo 18)

```
bash
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

Sans ça, Hadoop ne sait pas où Java est installé et refuse de démarrer. On a d'abord eu l'erreur
`ERROR: JAVA_HOME is not set` avant de corriger.

core-site.xml (diapo 19)

xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
    <!-- Toutes les commandes "hdfs dfs -..." se connectent ici -->
  </property>
  <property>
    <name>user.name</name>
    <value>hadoop</value>
  </property>
</configuration>
```

fs.defaultFS = l'adresse du NameNode. Quand tu tapes `hdfs dfs -ls /`, Hadoop se connecte à `localhost:9000` pour obtenir la liste des fichiers. C'est comme l'adresse d'un serveur FTP.

hdfs-site.xml (diapos 20-21)

xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value> <!-- 1 car mode pseudo-distribué -->
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:/usr/local/hadoop-3.3.6/hdfs/namenode</value>
    <!-- Où le NameNode stocke ses métadonnées -->
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:/usr/local/hadoop-3.3.6/hdfs/datanode</value>
    <!-- Où le DataNode stocke les blocs de données -->
  </property>
</configuration>
```

mapred-site.xml (diapo 23)

xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
    <!-- On dit à Hadoop d'utiliser YARN pour exécuter les jobs MapReduce -->
  </property>
</configuration>
```

Sans cette ligne, MapReduce s'exécute en mode local (une seule machine, sans YARN). Avec `yarn`, les jobs sont distribués sur le cluster et gérés par le ResourceManager.

yarn-site.xml (diapo 25)

```
xml
<configuration>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>localhost</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
    <!-- Active le Shuffle g  r   par YARN -->
  </property>
  <property>
    <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
    <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  </property>
</configuration>
```

mapreduce_shuffle active le service de Shuffle dans YARN. Sans   a, les donn  es ne peuvent pas transiter des Mappers vers les Reducers.

SSH sans mot de passe (diapos 15-16)

Hadoop doit pouvoir lancer des processus sur d'autres machines **sans demander de mot de passe**.

```
bash
# G  n  rer la paire de cl  s RSA
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa

# Copier la cl   publique dans authorized_keys
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

# Test : doit se connecter sans mot de passe
ssh localhost
```

La cl   priv  e reste sur la machine. La cl   publique est copi  e sur toutes les machines du cluster. Quand `start-dfs.sh` essaie de d  marrer le DataNode via SSH, l'authentification se fait par cl   → pas de mot de passe n  cessaire.

Formatage HDFS (diapo 29)

```
bash
hdfs namenode -format
...
```

   faire ****UNE SEULE FOIS**** lors de la premi  re installation.   a initialise le syst  me de fichiers HDFS, comme formater un disque dur. Si on le refait apr  s, on perd toutes les donn  es HDFS.

PARTIE 5 — MapReduce & Index Inversé (Code Java)

Qu'est-ce que MapReduce ? (diapo 24)

MapReduce est le **modèle de calcul distribué** d'**Hadoop**. L'idée : diviser un problème énorme en sous-problèmes traités **en parallèle** sur plusieurs machines. Il y a **3** phases :

```
| Phase | Qui ? | Quoi ? |
|---|---|---|
| **MAP** | Mapper — tu codes | Lit chaque ligne, extrait des paires (clé, valeur) |
| **SHUFFLE** | Hadoop — automatique | Trie et regroupe toutes les valeurs par clé |
| **REDUCE** | Reducer — tu codes | Traite chaque groupe, produit le résultat final |
```

****Points forts de MapReduce :****

- Les calculs se font **près des données** (Data Locality) → moins de trafic réseau
- Adapté au **traitement batch** (grands volumes, pas en temps réel)
- **Tolérant aux pannes** : si un Mapper échoue, Hadoop le relance automatiquement

Concept de l'Index Inversé (diapos 38-41)

Un **index inversé** c'est exactement ce que fait Google : pour chaque mot, on sait dans quelles pages (ou lignes) il apparaît.

Notre fichier `documents.txt` :

...

```
ligne1 : hadoop est un framework distribue
ligne2 : hadoop permet le traitement de big data
ligne3 : big data est important pour les entreprises
ligne4 : le traitement distribue est efficace
...
```

Résultat attendu de l'index inversé :

...

```
hadoop  ligne1, ligne2
big     ligne2, ligne3
data    ligne2, ligne3
est     ligne1, ligne3, ligne4
distribue ligne1, ligne4
traitement ligne2, ligne4
```

Le code Mapper — Analyse complète (diapo 40)

```
java
import java.io.IOException;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class InvertedIndexMapper
    extends Mapper<LongWritable, Text, Text, Text> {
    //      ↑clé entrée  ↑val entrée ↑clé sortie ↑val sortie

    private Text word = new Text();
```



```

private Text lineNumber = new Text();

@Override
public void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {

    String line = value.toString(); // contenu de la ligne
    String lineNum = "ligne" + (key.get() + 1); // ex: ligne1, ligne2...
    String[] words = line.split("\\s+"); // découpe par espaces

    for (String w : words) {
        w = w.toLowerCase().trim(); // normalise le mot
        word.set(w);
        lineNumber.set(lineNum);
        context.write(word, lineNumber); // ÉMET (mot, ligneN)
    }
}
}
...

```

****Explication de chaque partie :****

`extends Mapper<LongWritable, Text, Text, Text>` — Les 4 types génériques sont : type clé entrée, type valeur entrée, type clé sortie, type valeur sortie. `LongWritable` est un entier **long** (offset en octets dans le fichier). `Text` est une chaîne de caractères **Hadoop**.

`key.get() + 1` — La clé d'entrée du **Mapper** est l'offset en octets (position dans le fichier, pas un numéro de ligne). **On** l'utilise ici pour simuler un numéro de ligne lisible.

`line.split("\\s+")` — Découpe la ligne en tableau de mots selon les espaces. `\\s+` = un ou plusieurs espaces/tabulations.

`context.write(word, lineNumber)` — C'est l'instruction qui ****émet la paire****. **Hadoop** collecte toutes ces paires de tous les **Mappers** et les envoie au **Shuffle**.

****Ce que le Mapper produit pour ligne1 :****

```

...
(hadoop, ligne1)
(est, ligne1)
(un, ligne1)
(framework, ligne1)
(distribue, ligne1)
...

```

Le Shuffle — Automatique (diapo 43)

****Le Shuffle est la partie magique d'Hadoop. Tu n'écris AUCUN code pour cette étape.****

Hadoop collecte tous les résultats de tous les **Mappers**, les ****trie alphabétiquement**** par clé, puis ****regroupe toutes les valeurs de la même clé**** ensemble avant de les envoyer au **Reducer**.

...

Sortie Mapper brute : **Après Shuffle (groupé) :**

hadoop → ligne1	est → [ligne1, ligne3, ligne4]
est → ligne1	hadoop → [ligne1, ligne2]
un → ligne1	un → [ligne1]
hadoop → ligne2	...
est → ligne3	

Les 4 opérations du Shuffle (aucune n'est codée par toi) :

1. Tri alphabétique des clés
 2. Regroupement de toutes les valeurs par clé
 3. Transfert vers le bon Reducer
 4. Merge des données de plusieurs Mappers
-

Le code Reducer — Analyse complète (diapo 41)

```
java
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class InvertedIndexReducer
    extends Reducer<Text, Text, Text, Text> {
    //      ↑clé entrée ↑val entrée ↑clé sortie ↑val sortie

    @Override
    public void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        List<String> lines = new ArrayList<>();

        for (Text val : values) {      // val = "ligne1", "ligne2", etc.
            if (!lines.contains(val.toString())) { // évite les doublons
                lines.add(val.toString());
            }
        }

        // Émet : (mot, "ligne1, ligne2, ...")
        context.write(key, new Text(String.join(", ", lines)));
    }
    ...

    /**Explication : **

    `Iterable<Text> values` — Reçoit la liste de toutes les lignes pour ce mot (c'est le résultat du Shuffle). Pour
    `hadoop`, values = `["ligne1", "ligne2"]`.

    `!lines.contains(...)` — Un mot peut apparaître deux fois sur la même ligne. On évite les doublons dans
    l'index.

    `String.join(", ", lines)` — Construit la chaîne finale `"ligne1, ligne2"`.

    /**Ce que le Reducer produit : **
    ...

    hadoop → ligne1, ligne2
    est    → ligne1, ligne3, ligne4
    big    → ligne2, ligne3
```

Le Driver Java — Le chef d'orchestre (diapo 42)

```
java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class InvertedIndexDriver {
    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Inverted Index");

        job.setJarByClass(InvertedIndexDriver.class); // classe principale du JAR
        job.setMapperClass(InvertedIndexMapper.class); // qui est le Mapper
        job.setReducerClass(InvertedIndexReducer.class); // qui est le Reducer

        job.setMapOutputKeyClass(Text.class); // type clé sortie Mapper
        job.setMapOutputValueClass(Text.class); // type val sortie Mapper
        job.setOutputKeyClass(Text.class); // type clé résultat final
        job.setOutputValueClass(Text.class); // type val résultat final

        FileInputFormat.addInputPath(job, new Path(args[0])); // dossier HDFS input
        FileOutputFormat.setOutputPath(job, new Path(args[1])); // dossier HDFS output

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Le Driver **configure et lance** le job. Il dit à Hadoop : "utilise ce Mapper, ce Reducer, lis depuis ce dossier HDFS, écris le résultat là." C'est le point d'entrée du programme.

Compilation et lancement (diapo 42)

```
bash
# 1. Compiler les 3 fichiers Java
javac -classpath $(hadoop classpath) \
    -d ~/inverted_index/classes \
    InvertedIndexMapper.java \
    InvertedIndexReducer.java \
    InvertedIndexDriver.java

# 2. Créer l'archive JAR
jar -cvf ~/inverted_index/inverted_index.jar \
    -C ~/inverted_index/classes .

# 3. Uploader le fichier texte dans HDFS
hdfs dfs -mkdir -p /user/hadoop/input
hdfs dfs -put ~/documents.txt /user/hadoop/input/

# 4. Lancer le job MapReduce
hadoop jar inverted_index.jar InvertedIndexDriver \
```

```
/user/hadoop/input /user/hadoop/output
```

```
# 5. Lire les résultats
```

```
hdfs dfs -cat /user/hadoop/output/part-r-00000
```

Pourquoi `$(hadoop classpath)` ? Hadoop a des dizaines de fichiers JAR nécessaires pour compiler. Cette commande les récupère automatiquement tous.

Le fichier de sortie s'appelle `part-r-00000` car Hadoop peut avoir plusieurs Reducers en parallèle. Chaque Reducer écrit son propre fichier : `part-r-00000`, `part-r-00001`, etc.

PARTIE 6 — Word Count (Compter les mots)

Concept (diapo 45)

Le Word Count est le **"Hello World" de MapReduce**. Au lieu de savoir *où* est un mot (index inversé), on compte *combien de fois* il apparaît.

Critère	Index Inversé	Word Count
Objectif	Où est le mot (lignes)	Combien de fois apparaît-il
Mapper émet	(mot, numéro_ligne)	(mot, 1)
Valeur Mapper	Text (ex: "ligne1")	IntWritable (toujours 1)
Shuffle regroupe	mot → [ligne1, ligne2]	mot → [1, 1, 1, ...]
Reducer fait	Construit liste de lignes	Additionne les 1
Résultat	hadoop → ligne1, ligne2	hadoop → 2
Usage réel	Moteur de recherche	Statistiques de fréquence

Le Mapper Word Count (diapo 46)

```
java
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {
    //      clé sortie val sortie = ENTIER

    private Text word = new Text();
    private final static IntWritable one = new IntWritable(1); // TOUJOURS 1

    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();
        String[] words = line.split("\\s+");
```

```

    for (String w : words) {
        w = w.toLowerCase().trim();
        if (!w.isEmpty()) {
            word.set(w);
            context.write(word, one); // ÉMET (mot, 1) — valeur TOUJOURS 1
        }
    }
}
}
}
...

```

****La différence clé avec l'Index Inversé :** la valeur émise est `IntWritable(1)` (un entier) et non du texte. La valeur est toujours 1 car le **Reducer** additionne tous les 1.

Paires émises pour les deux premières lignes :

```

(hadoop, 1) (est, 1) (un, 1) (framework, 1) (distribue, 1)
(hadoop, 1) (permet, 1) (le, 1) (traitement, 1) (big, 1) (data, 1)

```

Le Reducer Word Count (diapo 47)

```

java
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0; // compteur d'occurrences

        for (IntWritable val : values) { // val = 1, 1, 1, ...
            sum += val.get();           // 1 + 1 + 1 = total
        }

        context.write(key, new IntWritable(sum)); // ÉMET (mot, total)
    }
}
...

```

Pour `hadoop`, le **Reducer** reçoit `[1, 1]` et fait `1 + 1 = 2`. Pour `est`, il reçoit `[1, 1, 1]` et fait `1 + 1 + 1 = 3`. Simple et massivement parallélisable.

Résultats du Word Count (diapo 48)

```

...
big      2
data     2
distribue 2
efficace 1
entreprises 1

```

```

est      3
framework 1
hadoop   2
important 1
le       2
permet   1
pour     1
traitement 2
un       1
...

```

****Détail du Shuffle pour Word Count :****
 ...

Après Mapper :	Après Shuffle :
(hadoop, 1) 	hadoop → [1, 1] → sum = 2
(hadoop, 1) 	est → [1, 1, 1] → sum = 3
(est, 1) 	big → [1, 1] → sum = 2
(est, 1) 	un → [1] → sum = 1
(est, 1) 	

PARTIE 7 — Interfaces Web de Surveillance

Différence entre localhost:50070 et localhost:8088 (diapo 37)

URL	Interface	Ce qu'elle montre
localhost:50070	NameNode HDFS	État du système de fichiers : DataNodes connectés, espace disque, blocs stockés
localhost:8088	ResourceManager YARN	État des jobs : applications soumises, ressources CPU/mémoire allouées, nœuds de calcul

localhost:50070 (diapos 31-34) : Montre que le cluster HDFS est sain — 1 DataNode actif, 61 GB d'espace disponible, aucune erreur, heartbeat toutes les 1 seconde.

localhost:8088 (diapos 35-36 et 49) : Montre les jobs MapReduce soumis. La diapo 49 montre 2 applications FINISHED/SUCCEDED : Inverted Index (~1 min) et Word Count (~39 sec).

PARTIE 8 — Guide diapo par diapo

Diapo	Ce qu'il faut dire
1 (Couverture)	"On va voir comment configurer Hadoop 3.3.6 de A à Z et l'utiliser pour du calcul distribué avec MapReduce."
2 (Intro Big Data)	Annonce le plan en 4 parties.
3 (Pourquoi Big Data ?)	Pose la question des emails. Explique les 5V avec des exemples.
4 (Limites trad.)	Analogie du bibliothécaire. Single Point of Failure = un seul serveur, une seule panne = tout s'arrête.

Diapo	Ce qu'il faut dire
6 (Définition Hadoop)	"Diviser pour régner." Les 3 piliers : HDFS stocke, YARN gère, MapReduce calcule.
7 (Pourquoi Hadoop ?)	Scalabilité horizontale vs verticale. Commodity hardware = serveurs pas chers.
8 (Netflix/Facebook)	Ces exemples rendent Hadoop concret et motivant.
10-11 (Config requise)	Linux obligatoire : performance I/O, SSH natif, compatibilité native Hadoop.
15-16 (SSH)	Pourquoi SSH sans mot de passe est indispensable pour les clusters multi-machines.
18 (hadoop-env.sh)	"On dit à Hadoop où trouver Java. Sans ça, rien ne démarre."
19 (core-site.xml)	"fs.defaultFS = l'adresse du NameNode. Comme l'adresse d'un serveur FTP."
20-22 (hdfs-site.xml)	Répertoires de stockage + réplication. Expliquer pourquoi 3 réplications.
23 (mapred-site.xml)	"mapreduce.framework.name=yarn : on force l'utilisation de YARN au lieu du mode local."
26-27 (YARN)	ResourceManager + NodeManager + ApplicationMaster. Analogie du chef de chantier.
29 (Format HDFS)	"À faire UNE SEULE FOIS. Comme formater un disque dur."
30 (jps)	Expliquer chaque processus. NameNode = cerveau, DataNode = muscles, ResourceManager = chef.
31-36 (Interfaces web)	Montrer les deux interfaces et leur différence.
39 (Charger fichier)	"On met un fichier texte dans HDFS avec hdfs dfs -put. C'est l'entrée de notre job."
40 (Mapper Index)	Montrer le code. Dire : "Pour chaque mot de chaque ligne, on crie (mot, numéroLigne)."
43 (Shuffle)	"Aucun code. Hadoop le fait automatiquement. Il trie et regroupe par clé."
41 (Reducer Index)	"On reçoit le groupe. On construit la liste des lignes. On émet le résultat."
42 (Driver + compil)	Montrer les 3 commandes : javac, jar, hadoop jar. Le Driver = point d'entrée.
44 (Récap flux)	Résumé visuel. 3 fichiers Java → 4 lignes traitées → 13 mots indexés.
45-47 (Word Count code)	"Même pattern. Seule différence : la valeur est 1 et le Reducer additionne."
48 (Résultats WC)	Lire le tableau. Expliquer le Shuffle avec hadoop → [1,1] → sum = 2.
49 (Vérif YARN)	"La preuve que tout marche : 2 jobs SUCCEEDED dans YARN."
50 (Comparaison finale)	Lire le tableau. Conclure : même architecture MapReduce, logique du Reducer différente.
51 (Récap final)	Flux complet : HDFS Input → Mapper → Shuffle → Reducer → HDFS Output. Questions ?

SYNTHÈSE — Tout Hadoop en une phrase par concept

Concept	En une phrase
Big Data	Données trop volumineuses pour les bases classiques → besoin de distribution
Hadoop	Framework open-source qui distribue stockage + calcul sur des machines ordinaires
HDFS	Découpe les fichiers en blocs de 128 Mo, les réplique 3× sur les DataNodes
NameNode	Cerveau de HDFS : stocke les métadonnées (où est quoi), PAS les données
DataNode	Muscles de HDFS : stocke physiquement les blocs
SecondaryNameNode	Fusionne fsimage + edits pour alléger le NameNode (pas un backup !)
YARN	Chef de chantier : gère les ressources CPU/mémoire et distribue les tâches
ResourceManager	Patron de YARN : accepte les jobs, alloue les ressources globalement
NodeManager	Agent local : exécute les conteneurs (tâches Map/Reduce) sur sa machine
ApplicationMaster	Chef de projet d'un job : surveille ses tâches et les relance si nécessaire
MapReduce	Modèle de calcul : Map (extraire), Shuffle (regrouper auto), Reduce (synthétiser)
Mapper	Transforme chaque ligne en paires (clé, valeur)
Shuffle	Hadoop regroupe toutes les valeurs de même clé — automatique, aucun code
Reducer	Prend un groupe (clé + liste de valeurs) et produit le résultat final
Index Inversé	Mapper émet (mot, ligne) → Reducer liste les lignes → hadoop → ligne1, ligne2
Word Count	Mapper émet (mot, 1) → Reducer additionne → hadoop → 2
localhost:50070	Interface web NameNode HDFS — surveillance du stockage
localhost:8088	Interface web ResourceManager YARN — surveillance des jobs